Ingeniería Mecatrónica

Tecnologías de Microprocesamiento

Laboratorio 3

Problema A

Control de temperatura con ATmega328P

Estudiante: _____

C.I.: _____

Docentes: Jesús Martínez y Marcelo Díaz

Fecha: _____

Semestre IV

1. Introducción

En este problema se realizó un sistema de control de temperatura usando el microcontrolador ATmega328P. El sensor LM35 mide la temperatura y el programa decide si debe encender el calefactor, mantener las salidas apagadas o hacer funcionar el ventilador a una de tres velocidades.

La temperatura, el punto medio y el estado del sistema se muestran en un LCD 16x2. También se envían datos por UART cada cinco segundos. Con esos datos se realizó una gráfica en Python para observar el comportamiento del sistema.

1.1. Objetivo

Implementar y comprobar un controlador de temperatura que integre las entradas y salidas principales del ATmega328P: conversión analógica, PWM, comunicación I2C, UART y memoria EEPROM.

1.2. Materiales principales

Componente	**Uso**
Arduino UNO R3 con ATmega328P	Ejecuta el programa y permite cargarlo por USB.
Sensor LM35	Mide la temperatura ambiente.
LCD 16x2 con módulo I2C	Muestra temperatura, punto medio y estado.
Motor DC	Representa el ventilador.
MOSFET IRLZ44N	Controla el motor con la señal PWM.
Diodo 1N4007	Protege el circuito de los picos producidos por el motor.
LED rojo y resistencia de 220 Ω	Representan el calefactor.
Resistencias de 330 Ω y 10 kΩ	Conexión y apagado seguro del Gate del MOSFET.
Fuente externa regulada	Alimenta el motor sin exigir corriente al Arduino.
Protoboard y cables	Permiten realizar el armado.

2. Conexiones realizadas

Las conexiones principales del montaje físico fueron las siguientes:

Elemento	**Pin**	**Conexión**
LM35	VOUT	A0 del Arduino
LCD I2C	SDA / SCL	A4 / A5
LED calefactor	Ánodo	D8 mediante 220 Ω
IRLZ44N	Gate	D6 mediante 330 Ω
IRLZ44N	Drain	Terminal negativo del motor
IRLZ44N	Source	GND común
Motor DC	Terminal positivo	Positivo de la fuente externa
Diodo 1N4007	Franja gris	Positivo del motor
Fuente externa	Negativo	GND común con el Arduino

El Arduino, el LM35 y el LCD se alimentaron desde el cable USB. El motor utilizó una fuente aparte. El negativo de esa fuente se unió al GND del Arduino para que ambos circuitos tuvieran la misma referencia. El motor no se conectó directamente al pin de 5 V del Arduino.

3. Funcionamiento del programa

El punto medio inicial es 22 °C. Los límites se calculan a partir de ese valor, por lo que cambian automáticamente cuando el usuario modifica el punto medio desde el menú UART.

Temperatura con PM = 22 °C	**Calefactor**	**Ventilador**
0 a 15 °C	Encendido	Apagado
16 a 28 °C	Apagado	Apagado
29 a 39 °C	Apagado	Velocidad baja
40 a 50 °C	Apagado	Velocidad media
51 a 100 °C	Apagado	Velocidad alta
Mayor a 100 °C	Apagado	Error de sensor

Las velocidades se generan por PWM en D6. Los valores usados son 89 para velocidad baja, 166 para velocidad media y 255 para velocidad alta, sobre una escala de 0 a 255.

4. Simulación en Proteus

Antes del armado físico se probó el circuito en Proteus. La simulación incluyó el ATmega328P, el LM35, el LCD con expansor I2C, el terminal UART, el calefactor y el control del motor.

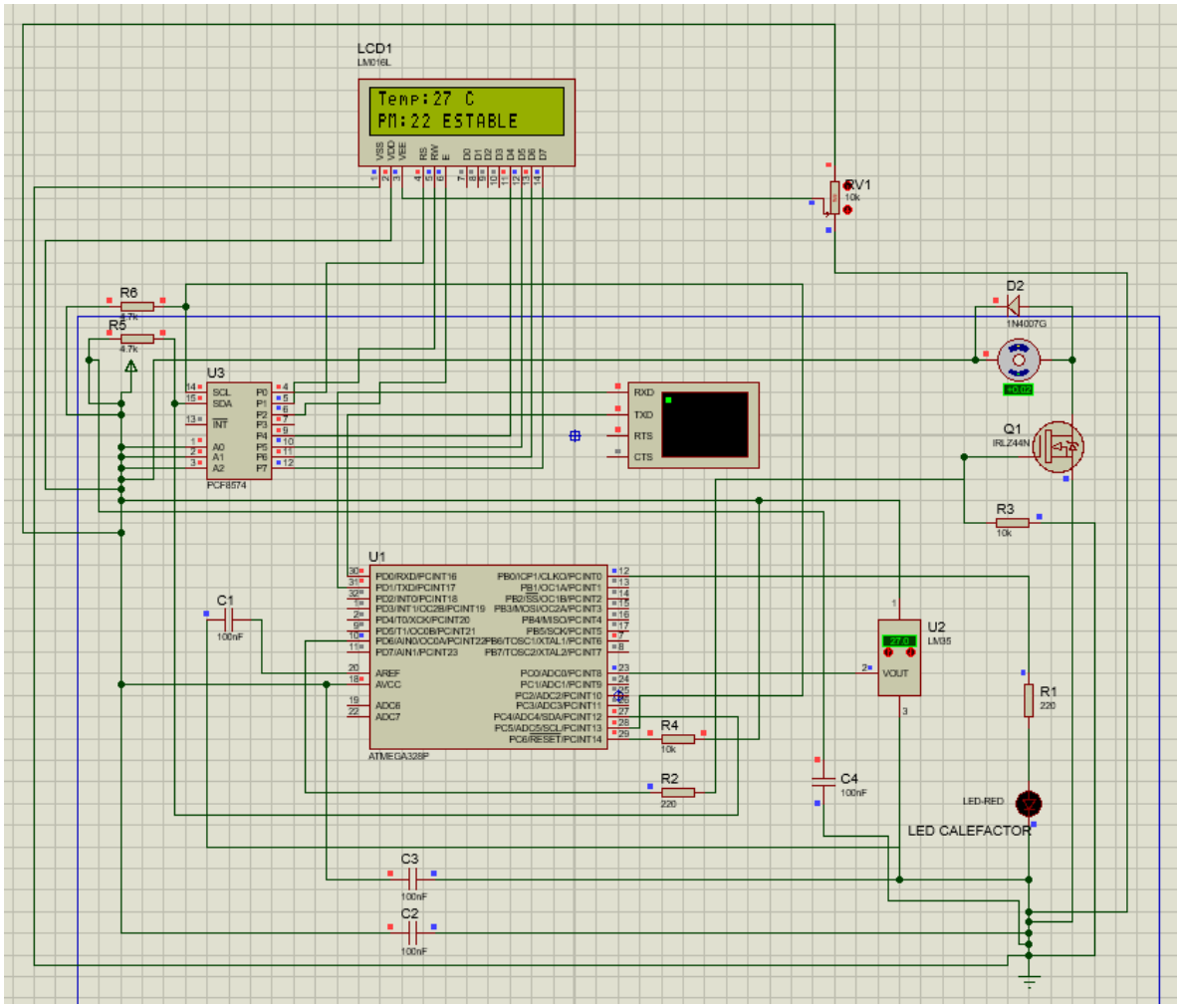


Figura 1: Simulación del Problema A funcionando en Proteus.

En la prueba mostrada se obtuvo una temperatura de 27 °C con un punto medio de 22 °C. Como el valor está dentro del rango de 16 a 28 °C, el LCD muestra el estado `ESTABLE` y las dos salidas permanecen apagadas.

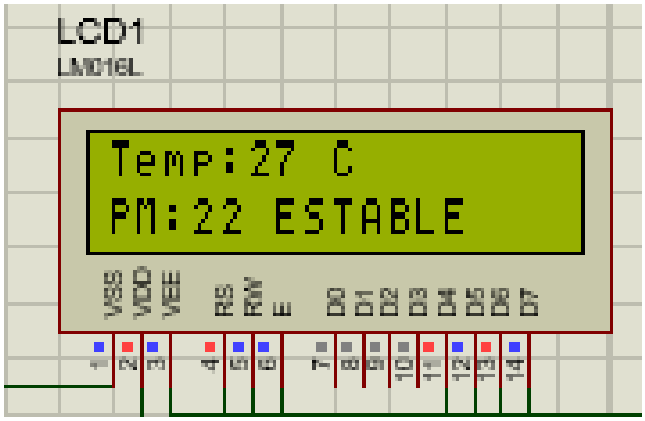


Figura 2: Información mostrada en el LCD durante la simulación.

5. Armado físico y comunicación

Luego de comprobar la simulación se realizó el montaje en protoboard. El motor se conectó mediante el MOSFET y una fuente externa, mientras que el LM35 y el LCD se alimentaron desde el Arduino.

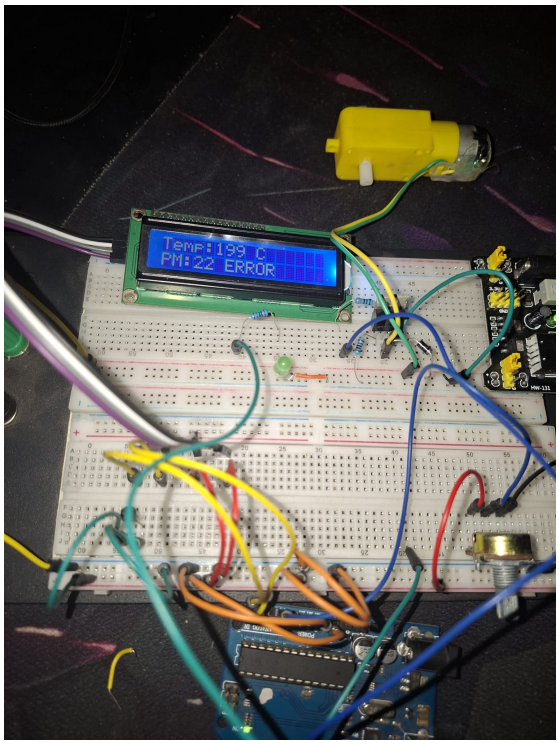


Figura 3: Montaje físico completo utilizado para las pruebas.

La fotografía fue tomada durante una prueba de lectura fuera del rango válido. Por ese motivo el LCD indica error. Esta condición hace que el programa apague el calefactor y el ventilador como medida de protección.

El menú UART permite aumentar el punto medio con la tecla 1, disminuirlo con la tecla 2, consultarlo con la tecla 3 y volver a mostrar el menú con la tecla M. El valor elegido queda guardado en la EEPROM.

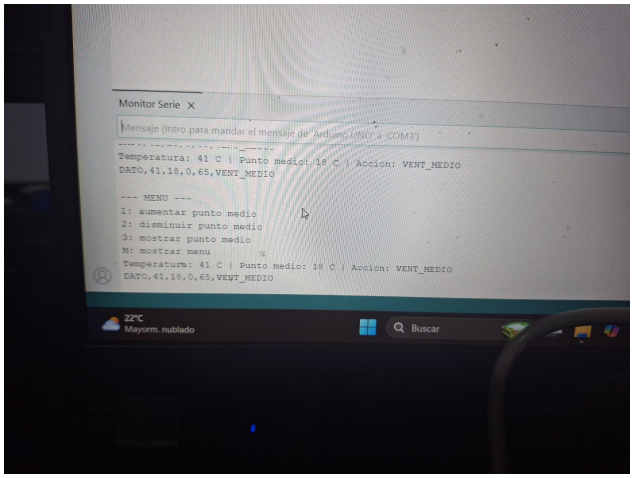


Figura 4: Menú y datos recibidos en el monitor serial a 9600 baudios.

6. Registro y visualización de resultados

Los datos enviados por UART se registraron con un programa en Python. Para esta prueba se cambió la entrada de temperatura de forma manual para recorrer los distintos rangos. Se guardaron 20 mediciones durante aproximadamente 104 segundos, manteniendo el punto medio en 22 °C.

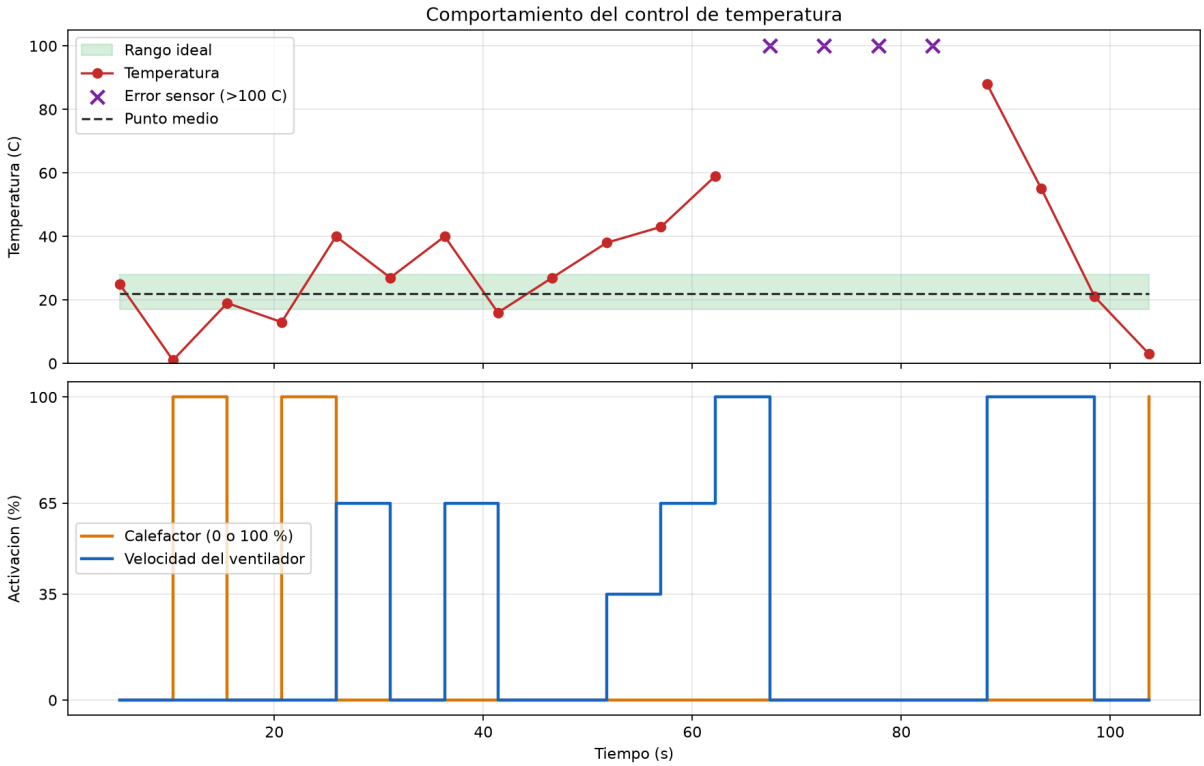


Figura 5: Registro de temperatura y acciones del sistema.

La parte superior muestra la temperatura medida, el punto medio y la zona ideal. Las lecturas mayores a 100 °C aparecen marcadas como error de sensor. La parte inferior muestra cuándo se encendió el calefactor y el porcentaje de velocidad del ventilador.

La prueba permitió comprobar que cada lectura produjo la acción esperada. Por ejemplo, a 13 °C se encendió el calefactor, a 38 °C se seleccionó velocidad baja, a 43 °C velocidad media y a 59 °C velocidad alta. Como las temperaturas fueron modificadas manualmente, esta gráfica comprueba la lógica del programa y no representa el calentamiento natural de una habitación.

7. Conclusión

Se logró implementar el control de temperatura pedido para el Problema A. El ATmega328P pudo leer el LM35, controlar el calefactor y el motor, mostrar los datos en el LCD y comunicarse con la computadora por UART. También se comprobó que el punto medio puede modificarse y que los rangos se ajustan a partir de ese valor. La simulación, el montaje físico y el registro en Python permitieron verificar el funcionamiento general del sistema.